

Data Quality & Validation Report

AIPI-561, Assignment 3

Overview

Production environments present a challenge to managing machine learning training and inference pipelines. In experimental contexts, we can iteratively explore, repair and apply datasets to optimize a model. A static, manually curated substrate gives way to a living distribution that must be managed with automation. Here we report on the automatic identification and remediation of data quality issues, as they pertain to our prior work deploying a taxi cab fare forecasting service.

Data Issues Identified

Our predictions are powered by a tabular dataset with largely numerical features. We employ a standard exploratory data analysis process to study the baseline and contrast with a ‘corrupted’ temporal slice which introduces some classic data quality challenges. Findings are enumerated here:

1. Trip Count (trip_count) range aberrations: the corrupted slice has both negative (353 rows) and improbable positive (311 rows over baseline max, frequently ‘9999’ or ‘99999’) values suggesting an upstream logic error or corruption. This feature is an input to virtually all of the services offered through our API endpoints and consequently risks inducing unexpected behavior.
2. Duplicate Rows: we see a non-trivial number of duplicate rows (8134) in the corrupted slice. These potentially skew the temporal averages we maintain for predictions. See more on this below.
3. Null Values: null values were present in the baseline dataset only and were not identified in the problematic slice
4. Pricing (cbd_pricing_active) distribution shift: the central business district pricing indicator represented both inactive (0) and active (1) states in the baseline, however the corrupt slice exclusively carries the active state. This was not deemed to be a risk to our predictions and was not remediated, but would be essential to characterize to appreciate output distributions of the model when tracking performance.

Continuous Validation

In this scenario, we simulate an actively growing dataset by monitoring a file committed to our git repository. In practice we are unlikely to commit new data to a repository in this manner, but it serves to test the strategy for continuous validation. Recent data in our dataset is arriving at a rate of roughly 150 inputs an hour. Our predictions leverage rolling averages starting on an hm basis. We elect to watch the data on an hourly basis accordingly to reduce the likelihood that these rolling averages will be undermined by invalid data.

Recovery

Erroneous trip_count values could be imputed (e.g. by using the mean of valid values) if we were short on data, but that is not the case here and this feature is also used as part of our rolling computations to support future predictions. We have a similar situation with duplicate rows: we would want to omit the redundant rows from our rolling computations due to the potential to skew the distribution. In both cases, our resolution is to simply filter (drop) the offending rows.